STOCKIQ · COMPUTAÇÃO EM NUVEM II

Nuvem AWS S3 e SQS

Documentação do que o StockIQ já usa de nuvem: armazenamento de fotos no S3, mensageria com SQS para o inventário, configuração, segurança, evidências e pendências.

Amazon S3 | Amazon SQS | IAM | Node.js · TypeScript | PostgreSQL

Gabriel da Silveira Pessoni · Lívia Portela Ferreira
FATEC Franca — Desenvolvimento de Software Multiplataforma · 6º semestre

Projeto Integrador VI
Computação em Nuvem II · Mineração de Dados

Sumário

01	**Visão geral e situação atual**	O que está pronto, o que falta
02	**Arquitetura**	Visão da nuvem e fluxo geral
03	**Armazenamento de objetos (Amazon S3)**	Upload de fotos de produto
04	**Mensageria (Amazon SQS)**	Ajuste de estoque do inventário
05	**Segurança**	Credenciais, acesso e dados
06	**Operação e execução**	Como rodar e como implantar
07	**Testes e evidências**	O que foi executado e com qual resultado
08	**Próximos passos**	Evolução da arquitetura de nuvem

01 Visão geral e situação atual

O StockIQ foi desenhado para rodar na AWS. Este documento descreve a parte de nuvem que **já está implementada no código** — o armazenamento de objetos (S3) e a mensageria assíncrona (SQS) —, sua configuração, segurança e as verificações realizadas.

2	1	202	26
SERVIÇOS AWS NO CÓDIGO	FILA + DLQ	TESTES PASSANDO	SUÍTES DE TESTE

Componente	Papel no sistema	Situação
Amazon S3	Fotos de produtos (upload pelo front-end, via API)	Implementado
Amazon SQS	Fila de processamento do ajuste de estoque do inventário	Implementado · worker e DLQ
IAM	Permissões mínimas para S3 e SQS	Políticas mínimas definidas
EC2 · Application Load Balancer	Hospedar API e front-end com balanceamento	Planejado
RDS / PostgreSQL em EC2	Banco de dados em nuvem	Planejado (hoje: PostgreSQL local)
CloudWatch	Métricas e logs centralizados	Planejado

02 Arquitetura

A API (Node.js/Express) é o único componente que fala com a AWS: o navegador nunca recebe credenciais. Fotos vão do navegador para a API e daí para o S3; o ajuste de estoque do inventário é publicado no SQS e executado por um worker.

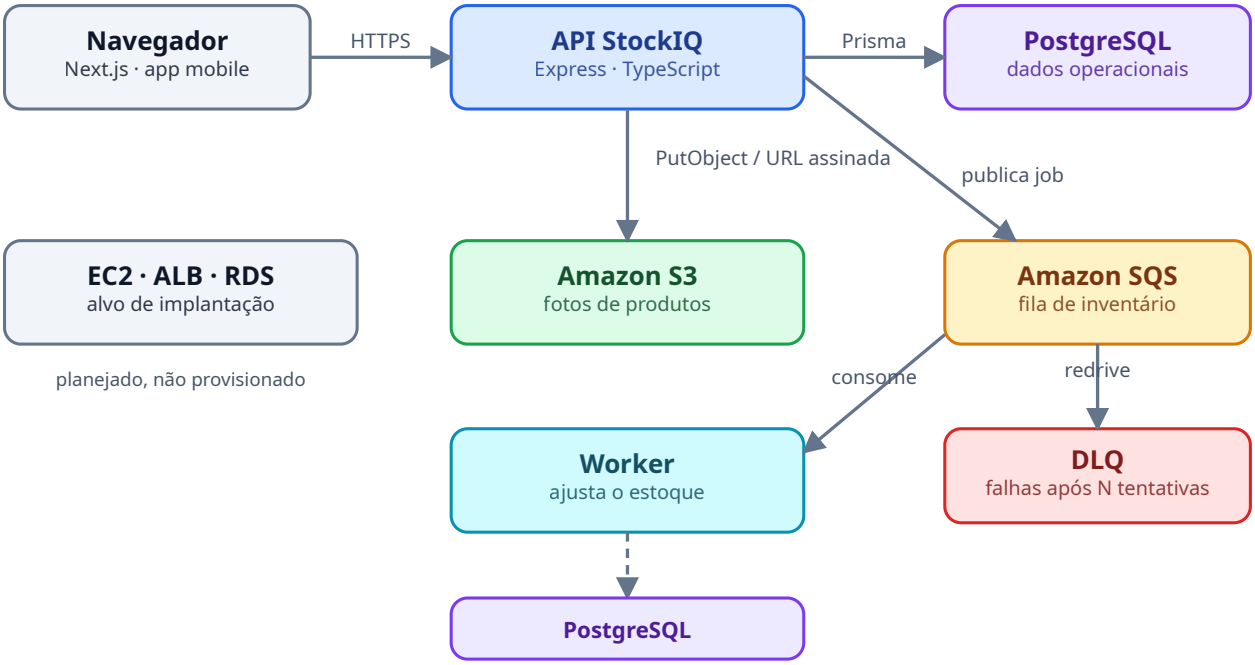


Figura 1 — Componentes de nuvem do StockIQ (verde/âmbar: implementado no código; cinza: planejado)

03 Armazenamento de objetos (Amazon S3)

O produto tem uma foto opcional. O arquivo é enviado ao endpoint `POST /api/products/:id/image` (campo multipart `image`); a API valida, grava no S3 e guarda apenas a **chave** do objeto no banco (`products.imageKey`). As respostas trazem `imageUrl` como **URL assinada temporária** (1 h por padrão), de modo que o bucket pode permanecer privado.

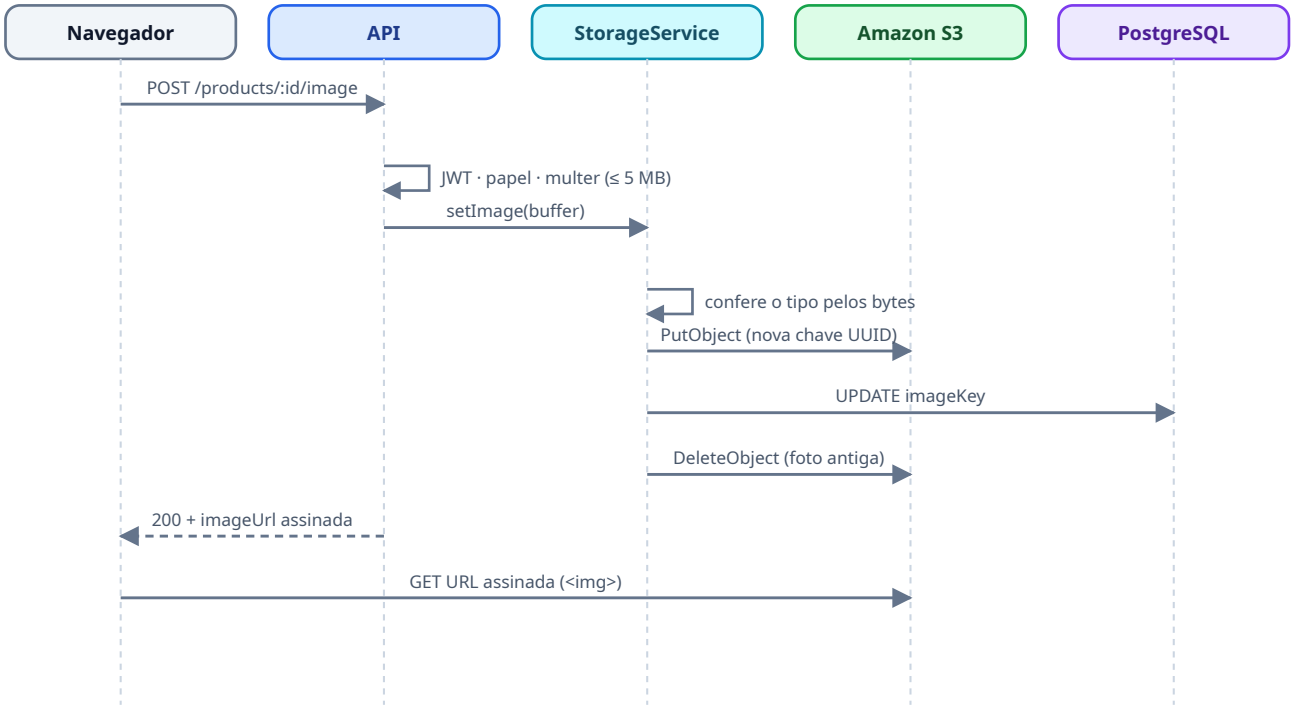


Figura 2 — Fluxo de upload de foto

Decisões de projeto

Tema	Como é feito	Por quê
Chave do objeto	`<prefixo>products/<idProduto>/<uuid>.<ext>`	UUID por envio: o conteúdo nunca muda (cache seguro) e não há colisão
Prefixo	`STORAGE_KEY_PREFIX` (padrão `stockiq/`)	Isola o projeto quando o bucket é compartilhado
Validação	JPEG, PNG, WebP ou GIF; tipo conferido pelos primeiros bytes; SVG recusado; limite `STORAGE_MAX_IMAGE_MB` (5)	O tipo declarado pelo cliente não é confiável; SVG pode conter script
Acesso	URL pré-assinada (`GetObject`), validade `STORAGE_URL_TTL_SECONDS`	Bucket privado; links expiram
Consistência	Sobe o novo → grava no banco (se falhar, apaga o novo) → só então apaga o antigo	Nunca fica um produto apontando para objeto inexistente
Órfãos	`npm run storage:cleanup` remove objetos sem produto, após `STORAGE_ORPHAN_GRACE_DAYS` (7)	Cobre quedas entre o upload e a gravação no banco
Auditoria	URLs assinadas são removidas do log de auditoria	Um link assinado funciona como senha temporária

Configuração

Variável	Descrição
`STORAGE_DRIVER`	`s3` (AWS) ou `local` (pasta em disco, só desenvolvimento)
`S3_BUCKET` · `S3_REGION`	Bucket e região do S3
`S3_ACCESS_KEY_ID` · `S3_SECRET_ACCESS_KEY`	Credenciais do usuário IAM (somente no `.env`, nunca versionadas)
`S3_SESSION_TOKEN`	Opcional: credenciais temporárias (ex.: AWS Academy)
`S3_ENDPOINT` · `S3_FORCE_PATH_STYLE`	Opcionais: S3 compatível (MinIO etc.)
`STORAGE_SIGNING_SECRET` · `STORAGE_LOCAL_DIR`	Driver local: segredo que assina as URLs de `/api/files` e pasta das fotos

Política IAM mínima

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::SEU-BUCKET/stockiq/*" },
    { "Effect": "Allow", "Action": "s3:ListBucket", "Resource": "arn:aws:s3:::SEU-BUCKET",
      "Condition": { "StringLike": { "s3:prefix": "stockiq/*" } } }
  ]
}
```

CREDENCIAIS

As chaves de acesso ficam apenas no `.env` do backend (ignorado pelo Git). Use um usuário/role exclusivo do projeto com a política acima e troque a chave se ela for exposta.

04 Mensageria (Amazon SQS)

Concluir um inventário exige aplicar ao estoque cada divergência contada — uma operação por item, que pode ser grande. Para que a requisição HTTP não espere por esse trabalho (nem concorra com as demais requisições), a conclusão **grava o estado `pending` e publica uma mensagem na fila**; um **worker** consome a mensagem e faz os ajustes em lotes. A API responde imediatamente e a tela acompanha o progresso.

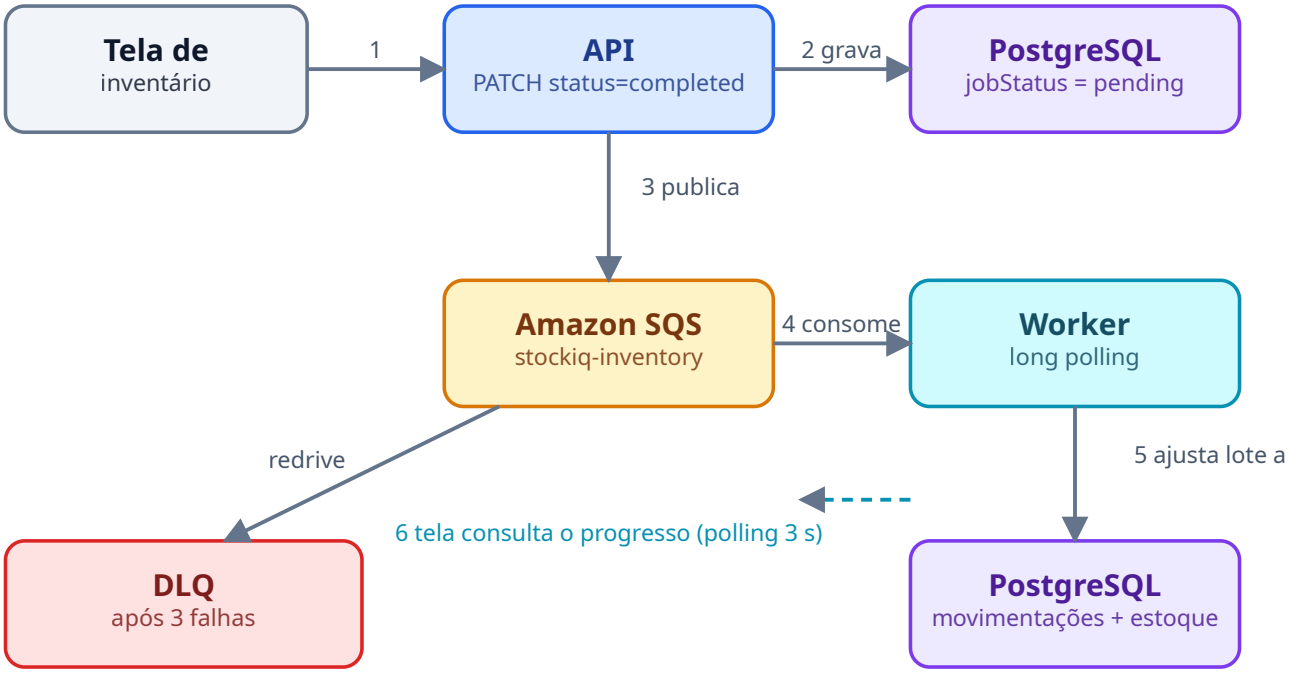


Figura 3 — Fluxo assíncrono do inventário

- **Estados do job**

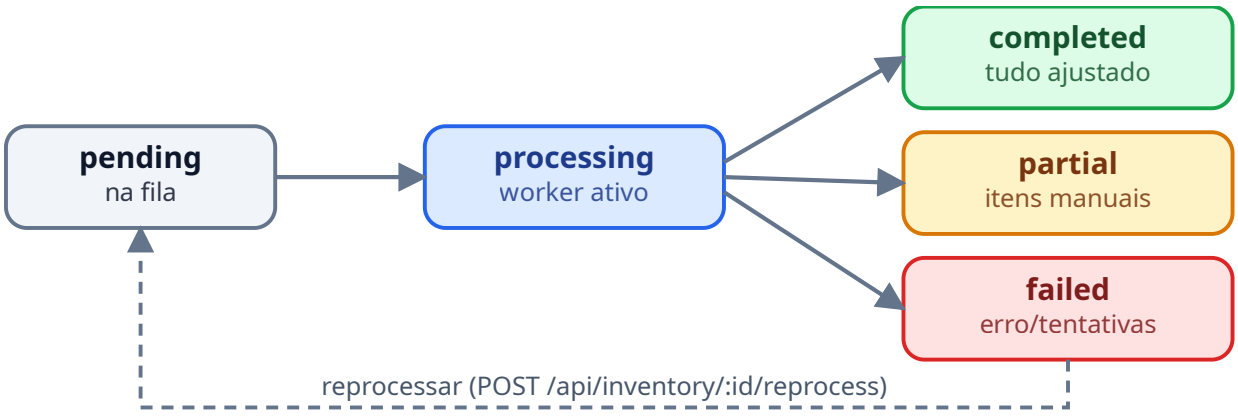


Figura 4 — Máquina de estados do processamento do inventário

Garantias e tratamento de falhas

Situação	O que acontece
Mensagem entregue mais de uma vez (SQS entrega *pelo menos uma vez*)	O processamento é **idempotente**: cada item só é ajustado se `adjustedAt` for nulo, e essa marca é gravada na mesma transação da movimentação
Erro de regra em um item (ex.: produto em vários endereços ou lotes)	O item vira pendência manual (`adjustmentError`); o job termina como `partial` e não trava os demais
Erro inesperado (ex.: banco fora do ar)	A mensagem **não** é apagada: reaparece após o prazo de visibilidade e é tentada de novo
Tentativas esgotadas (`QUEUE_MAX_RECEIVES` = 3)	O job vira `failed` (visível na tela) e a fila move a mensagem para a **DLQ**
Mensagem malformada	É descartada para não bloquear a fila
Fila indisponível ao concluir	A conclusão fica salva; o job é marcado `failed` e pode ser reprocessado
Worker cai no meio do job	A mensagem volta à fila; itens já ajustados são ignorados

Onde roda o worker

Por padrão a própria API consome a fila (`QUEUE_WORKER=embedded`), o que simplifica o desenvolvimento. Em produção, o worker deve rodar em **processo separado** (`npm run worker`), com `QUEUE_WORKER=off` na API — assim o processamento pesado não compete com as requisições e pode ser escalado com vários consumidores concorrentes.

Configuração

Variável	Descrição
`QUEUE_DRIVER`	`sqs` ou `local` (fila em memória: só desenvolvimento/testes)
`SQS_INVENTORY_QUEUE_URL`	URL da fila (impressa por `npm run queues:setup`)
`SQS_REGION` · `SQS_ACCESS_KEY_ID` · `SQS_SECRET_ACCESS_KEY` · `SQS_SESSION_TOKEN`	Por padrão reaproveitam as variáveis `S3_*`
`QUEUE_VISIBILITY_TIMEOUT_SECONDS` (120) · `QUEUE_MAX_RECEIVES` (3)	Prazo de visibilidade e tentativas antes da DLQ (igual ao `maxReceiveCount` da fila)
`QUEUE_WORKER` · `INVENTORY_BATCH_SIZE` (25)	Onde roda o worker e itens por lote (o progresso é atualizado a cada lote)
`SQS_QUEUE_PREFIX` (stockiq)	Nome das filas: `<prefixo>-inventory` e `<prefixo>-inventory-dlq`

`npm run queues:setup` cria a fila principal (long polling de 10 s, redrive para a DLQ) e a DLQ (retenção de 14 dias), de forma idempotente, e imprime as variáveis a copiar para o `.env` .

Permissões IAM

Quem	Ações SQS necessárias
Aplicação (API e worker)	`sqs:SendMessage` · `sqs:ReceiveMessage` · `sqs:DeleteMessage` · `sqs:GetQueueAttributes` na fila de inventário
Setup único (`queues:setup`)	`sqs:CreateQueue` · `sqs:GetQueueUrl` · `sqs:GetQueueAttributes` · `sqs:SetQueueAttributes`

05 Segurança

Ponto	Prática adotada
Credenciais	Somente no `.env` (no `.gitignore`); o código lê variáveis de ambiente e nunca as imprime. Em produção, prefira *role* da instância (EC2) em vez de chaves
Menor privilégio	Políticas por prefixo (`stockiq/*`) e por fila; separar o usuário do setup do usuário da aplicação
Credenciais temporárias	Suporte a `*_SESSION_TOKEN` (AWS Academy / STS)
Acesso às fotos	Bucket privado + URL assinada com validade curta; nenhuma URL assinada é gravada na auditoria
Upload	Autenticação JWT, papel admin/supervisor, limite de tamanho, tipo conferido pelo conteúdo, SVG recusado
Mensagens	Corpo validado por schema (Zod); mensagens inválidas são descartadas; o usuário solicitante vem na mensagem para a trilha de movimentação

06 Operação e execução

Desenvolvimento local (sem depender da AWS)

```
# banco descartável e fila SQS compatível
docker run -d --rm -p 127.0.0.1:55432:5432 -e POSTGRES_PASSWORD=postgres -e
POSTGRES_DB=stockiq_test postgres:16
docker run -d --rm -p 127.0.0.1:9324:9324 softwaremill/elasticmq-native

cd backend
npx prisma migrate deploy                # aplica as 5 migrations
QUEUE_DRIVER=local npm run dev           # API + worker embutido (fila em memória)
```

Com a fila na AWS

```
npm run queues:setup                     # cria fila + DLQ e imprime as variáveis
# copie QUEUE_DRIVER=sqs e SQS_INVENTORY_QUEUE_URL para o .env
npm run worker                           # processo dedicado ao consumo (QUEUE_WORKER=off na API)
```

Implantação-alvo

1. API e worker em instâncias EC2 (processos separados, ambos com a *role* IAM do projeto);
2. front-end Next.js em instância própria atrás do Application Load Balancer (HTTPS via ACM);
3. PostgreSQL em RDS ou EC2 em subnet privada; apenas o ALB é público;
4. CloudWatch para métricas da fila (mensagens visíveis, idade da mais antiga, DLQ) e logs do worker.

07 Testes e evidências

Verificação feita em 19/10/2026 contra um PostgreSQL 16 descartável em Docker (separado do banco de desenvolvimento) e um servidor SQS compatível (ElasticMQ). Resultado da suíte completa: **202 de 202 testes passando em 26 suítes**.

Verificação	Como foi feita	Resultado
Migrations do zero	`prisma migrate deploy` em banco vazio (5 migrations)	Aplicadas
Driver S3	Comandos `PutObject` / `DeleteObject` /listagem simulados; URL pré-assinada gerada localmente (16 testes)	Passou
Fluxo de upload de foto	Ordem upload → banco → remoção da antiga, falha de banco, limites e tipos (12 testes)	Passou
Driver SQS	Contra ElasticMQ: envio, long polling, reentrega por visibilidade e DLQ (2 testes)	Passou
Fila local	Visibilidade, reentrega e DLQ simuladas (10 testes)	Passou
Processamento e worker	Idempotência, lotes, pendência manual, retry (23 testes)	Passou
Inventário ponta a ponta	Concluir → fila → worker → estoque, com banco real; reprocessamento (4 testes)	Passou
Front-end	`tsc`, ESLint e `next build`	Passou

08 Próximos passos

Etapa	Ação	Resultado esperado
1	Bucket e usuário/role exclusivos do StockIQ, com as políticas mínimas deste documento	Isolamento e menor privilégio
2	API e worker em EC2 (processos separados) e banco em RDS	Ambiente de nuvem completo
3	Application Load Balancer, ACM e CloudWatch (métricas da fila e DLQ)	Disponibilidade e observabilidade
4	Eventos de movimentação para S3 (data lake) alimentando a mineração de dados	Análise desacoplada do banco transacional